

Expose a Kedro project as a REST API using FastAPI

Mental model

- **What becomes an endpoint?** A Kedro pipeline or a single node can be exposed as an HTTP endpoint. The request payload is validated with Pydantic, passed to a service function, which creates a **new** `KedroSession` and runs the selected pipeline. The output returned by `KedroSession.run()` contains any unregistered outputs as a Python dict ¹, which can be converted to JSON. Thin FastAPI controllers simply call into the service layer; they don't contain business logic or direct Kedro calls.
- **What should NOT happen in an endpoint?** FastAPI endpoints must remain light: they should not perform heavy computations, network I/O, or long-running Kedro jobs synchronously. The Zyneto best-practices article recommends off-loading heavy work (PDF generation, AI inference, etc.) out of the request cycle ². Controllers should only perform basic validation and routing. Running a Kedro pipeline can be CPU-bound, so consider moving long pipelines to background tasks or a queue when needed.
- **Kedro sessions and context** `KedroSession` manages the lifecycle of a run. The official docs show that when creating a session outside of the CLI you must first call `bootstrap_project(project_root)`, then use `KedroSession.create()` as a context manager ³. A session has a one-to-one mapping with a run; `KedroSession.run()` can be called **only once per session**, otherwise it raises `KedroSessionError` ⁴. Therefore the API must create a new session for every request (or at least for every run).
- **Thin API, thick service** Clean Architecture guidance stresses that controllers should be **thin**, containing no logic beyond input validation and routing. Business logic and Kedro interactions live in a separate service layer that can be unit-tested independently and reused outside of FastAPI. Pydantic models enforce type safety and validation ⁵; they should not embed business rules. Use `jsonable_encoder` to convert complex objects (like NumPy arrays or datetimes) to JSON serialisable structures ⁶.

Minimal working example

Assume a typical Kedro `src/` layout. The example below implements a simple pipeline that multiplies each number by a `scale` parameter. The API exposes two endpoints: `GET /health` and `POST /predict`. Replace `acme_kedro` with your package name.

pyproject.toml (snippet)

```
[tool.poetry]
name = "acme-kedro"
version = "0.1.0"

[tool.poetry.dependencies]
python = "^3.10"
kedro = "^0.19.0"
fastapi = "^0.110.0"
uvicorn = "^0.27.0"
pydantic = "^2.0.0"

[tool.poetry.group.test.dependencies]
pytest = "^8.0.0"
httpx = "^0.25.0"
```

requirements.txt (alternative)

```
kedro==0.19.11
fastapi
uvicorn
pydantic
pytest
httpx
```

src/acme_kedro/api/schemas.py

```
from typing import List
from pydantic import BaseModel, Field

class PredictRequest(BaseModel):
    data: List[float] = Field(..., description="List of input numbers")

class PredictResponse(BaseModel):
    predictions: List[float]

class HealthResponse(BaseModel):
    status: str
    version: str
```

src/acme_kedro/api/service.py

```
from pathlib import Path
from typing import List, Optional
import logging

from fastapi.encoders import jsonable_encoder
from kedro.framework.startup import bootstrap_project
from kedro.framework.session import KedroSession

# Locate the project root (three levels up from this file)
PROJECT_ROOT = Path(__file__).resolve().parents[3]

# Bootstrap the project so Kedro can find conf/base and src/
bootstrap_project(PROJECT_ROOT) # required for non-CLI usage 3

logger = logging.getLogger(__name__)

def predict(data: List[float], request_id: Optional[str] = None) -> List[float]:
    """Run the Kedro pipeline and return predictions.

    `extra_params` injects runtime values into the `parameters` dataset. A new
    session is created per request to honour Kedro's one-run-per-session rule
    4 .
    """
    extra_params = {"input_data": data}
    if request_id:
        logger.info("Running prediction request_id=%s", request_id)
    with KedroSession.create(project_path=PROJECT_ROOT,
        extra_params=extra_params) as session:
        output = session.run(pipeline_name="simple")
        predictions = output.get("predictions", [])
        return jsonable_encoder(predictions) # ensure JSON serialisable 6
```

src/acme_kedro/api/main.py

```
from fastapi import FastAPI, APIRouter, HTTPException

from acme_kedro import __version__
from .schemas import PredictRequest, PredictResponse, HealthResponse
from .service import predict

def create_app() -> FastAPI:
    app = FastAPI(title="Acme Kedro API", version=__version__)
```

```

router = APIRouter()

@router.get("/health", response_model=HealthResponse)
def health() -> HealthResponse:
    return {"status": "ok", "version": __version__}

@router.post("/predict", response_model=PredictResponse)
def run_prediction(request: PredictRequest) -> PredictResponse:
    try:
        preds = predict(request.data)
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc)) from exc
    return {"predictions": preds}

app.include_router(router)
return app

app = create_app()

```

src/acme_kedro/pipelines/simple/nodes.py

```

from typing import List

def predict_node(input_data: List[float], scale: float) -> List[float]:
    """Multiply each input by the scale parameter."""
    return [x * scale for x in input_data]

```

src/acme_kedro/pipelines/simple/pipeline.py

```

from kedro.pipeline import Pipeline, node
from .nodes import predict_node

PIPELINE_NAME = "simple"

def create_pipeline(**kwargs) -> Pipeline:
    return Pipeline([
        node(
            func=predict_node,
            inputs={"input_data": "params:input_data", "scale": "params:scale"},
            outputs="predictions",
            name="predict_node",

```

```
)  
])
```

src/acme_kedro/pipeline_registry.py

```
from typing import Dict  
from kedro.pipeline import Pipeline  
from acme_kedro.pipelines.simple import pipeline as simple_pipeline  
  
def register_pipelines() -> Dict[str, Pipeline]:  
    simple = simple_pipeline.create_pipeline()  
    return {  
        "simple": simple,  
        "__default__": simple,  
    }
```

conf/base/parameters.yml

```
scale: 2.0  
# `input_data` is passed dynamically via `extra_params` and is omitted here.
```

conf/base/catalog.yml

```
# This example leaves the catalog empty on purpose.  
# Outputs that aren't declared here (e.g. `predictions`) are returned  
# directly from `session.run()`1.
```

tests/test_api.py

```
from fastapi.testclient import TestClient  
from acme_kedro.api.main import app  
  
client = TestClient(app)  
  
def test_health() -> None:  
    resp = client.get("/health")  
    assert resp.status_code == 200  
    payload = resp.json()  
    assert payload["status"] == "ok"  
    assert "version" in payload
```

```
def test_predict() -> None:
    resp = client.post("/predict", json={"data": [1.0, 3.0]})
    assert resp.status_code == 200
    assert resp.json()["predictions"] == [2.0, 6.0]
```

📦Running locally

1. **Install dependencies.** Create a virtual environment and install requirements:

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

1. **Run the API.** Use Uvicorn to serve the FastAPI app. Uvicorn's `--reload` flag automatically reloads on code changes ⁷. The `--app-dir` flag tells Uvicorn where to find the package. From the project root:

```
uvicorn acme_kedro.api.main:app --reload --app-dir src
```

According to the Uvicorn documentation, starting the server is as simple as `uvicorn main:app --reload` ⁷; the additional flags shown above make it work with a `src/` layout.

1. **Try the endpoints.** In another terminal or using `curl`:

```
# Health check
curl -s http://127.0.0.1:8000/health

# Prediction
curl -s -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"data": [4, 5, 6]}'
```

The response will echo your inputs multiplied by the `scale` defined in `parameters.yml`.

1. **Run tests.** Use pytest to exercise the endpoints without starting a server:

```
pytest -q
```

❏ Kedro integration patterns (opinionated)

Create a session per request

`KedroSession` can only run a pipeline once per instance ⁴. Sharing a session across multiple API calls will cause `KedroSessionError`. Always wrap `KedroSession.create()` in a context manager within the service layer and do not store it globally.

Bootstrap the project

When you call Kedro programmatically (outside `kedro run`), call `bootstrap_project(project_root)` once to initialise configuration ³. Set `project_root` to the repository root (the folder containing `conf/` and `src/`). Omitting this step will cause `KedroSession` to raise `ModuleNotFoundError` when loading the context.

Select pipelines explicitly

The API can expose a single pipeline or multiple. Use `pipeline_name="my_pipeline"` in `session.run()` to pick which pipeline to execute. Register all available pipelines in `pipeline_registry.py`. This mirrors the CLI's `--pipeline` argument.

Pass runtime parameters

Use the `extra_params` argument of `KedroSession.create()` to inject request-specific values into the `parameters` dataset ³. In the example the request body becomes `input_data`, which the node accesses via `params:input_data`. Never mutate `conf/base/parameters.yml` at runtime.

Avoid writing to `data/`

HTTP requests should not persist intermediate data. Either:

1. **Return unregistered outputs.** If a node output isn't listed in `catalog.yml` it is returned directly from `session.run()` ¹. This is the approach used in the example.
2. **Use `MemoryDataset`.** For transient datasets you can define a `kedro.io.MemoryDataset` in the catalog. Load and save to/from memory; nothing touches the filesystem.

Logging and observability

Structured logging with a request ID helps trace logs across the pipeline. The FastAPI best-practices article recommends adding request IDs to logs for correlation ⁸. In the service layer you can accept a `request_id` and include it in log messages.

Keep controllers thin

The clean architecture example emphasises that controllers should perform only input validation and routing. All Kedro and business logic lives in the service layer. This separation makes testing easier and

prevents mixing web framework dependencies with domain code. Pydantic takes care of input validation ⁵. Avoid heavy operations in controllers; off-load long pipeline runs to background tasks or a message queue ².

Testing

FastAPI integrates seamlessly with `pytest` and `TestClient` ⁹. The example provides a small test suite (`tests/test_api.py`) that exercises both endpoints without starting Uvicorn. Run tests with:

```
pytest -q
```

Two tests cover the basics:

1. **Health** – sends a `GET /health` request and asserts that the status is `200` and the JSON contains `status: ok`.
2. **Predict** – posts a JSON payload to `/predict`, expects a `200` response and verifies that the output values equal the inputs multiplied by the `scale` parameter.

More tests can be added to handle invalid input, large payloads, or exceptions. Since the service layer returns plain Python types, you can test it directly without involving FastAPI.

Common pitfalls (and fixes)

- **Import path issues with `src/` layout.** When running Uvicorn from the repository root you must add `src` to `PYTHONPATH` or use `uvicorn --app-dir src`. Alternatively install the project in editable mode (`pip install -e .`).
- **Running outside the project root.** `KedroSession` resolves configuration relative to the current working directory. Always call `bootstrap_project(project_root)` and pass `project_path` to `KedroSession.create()` ³.
- **Thread safety.** Never share `KedroSession`, `DataCatalog` or Pipeline objects between threads. Create a fresh session per request ⁴. Avoid global mutable state; use dependency injection to pass services into controllers.
- **Returning Pandas/NumPy objects.** Pydantic cannot serialise DataFrames or arrays directly. Use `jsonable_encoder()` ⁶ or convert DataFrames with `df.to_dict(orient="records")` before returning. You can also define custom response models.
- **Long-running pipelines.** Avoid blocking the event loop with lengthy Kedro runs. Off-load heavy work to background tasks or an async queue (e.g. Celery, RQ) and return a job ID immediately ².

Cheatsheet

File layout (minimal)

```
acme_kedro/
├── pyproject.toml      # or requirements.txt
├── conf/
│   ├── base/
│   │   ├── catalog.yml
│   │   └── parameters.yml
├── src/acme_kedro/
│   ├── __init__.py
│   ├── api/
│   │   ├── main.py
│   │   ├── schemas.py
│   │   └── service.py
│   ├── pipelines/
│   │   ├── simple/
│   │   │   ├── nodes.py
│   │   │   └── pipeline.py
│   └── pipeline_registry.py
└── tests/
    └── test_api.py
```

Key commands

```
# Install dependencies and set up a virtualenv
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# Run the API locally
uvicorn acme_kedro.api.main:app --reload --app-dir src

# Run tests
pytest -q
```

Endpoint examples

```
curl -s http://127.0.0.1:8000/health
# => {"status":"ok","version":"0.1.0"}

curl -s -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
```

```
-d '{"data": [10, 20]}'  
# => {"predictions": [20.0, 40.0]}
```

Do / Don't

- **Do** call `bootstrap_project()` and create a new `KedroSession` per request ³.
- **Do** validate input with Pydantic and convert outputs with `jsonable_encoder()` ⁶.
- **Do** keep controllers thin and delegate to a service layer.
- **Don't** write to `data/` during API calls; use `MemoryDataset` or unregistered outputs.
- **Don't** reuse a `KedroSession` for multiple runs ⁴.
- **Don't** perform heavy, blocking work inside FastAPI endpoints ².

¹ ³ [kedro.framework.session.session.KedroSession — kedro 0.19.11 documentation](https://docs.kedro.org/en/0.19.11/api/kedro.framework.session.session.KedroSession.html)
<https://docs.kedro.org/en/0.19.11/api/kedro.framework.session.session.KedroSession.html>

² ⁸ [Best Practices in FastAPI Architecture](https://zyneto.com/blog/best-practices-in-fastapi-architecture)
<https://zyneto.com/blog/best-practices-in-fastapi-architecture>

⁴ [raw.githubusercontent.com](https://raw.githubusercontent.com/kedro-org/kedro/develop/kedro/framework/session/session.py)
<https://raw.githubusercontent.com/kedro-org/kedro/develop/kedro/framework/session/session.py>

⁵ [Building Robust Components with FastAPI and Pydantic - DEV Community](https://dev.to/kfir-g/building-robust-components-with-fastapi-and-pydantic-3f5f)
<https://dev.to/kfir-g/building-robust-components-with-fastapi-and-pydantic-3f5f>

⁶ [JSON Compatible Encoder - FastAPI](https://fastapi.tiangolo.com/tutorial/encoder/)
<https://fastapi.tiangolo.com/tutorial/encoder/>

⁷ [FastAPI - Uvicorn](https://www.tutorialspoint.com/fastapi/fastapi_uvicorn.htm)
https://www.tutorialspoint.com/fastapi/fastapi_uvicorn.htm

⁹ [Testing - FastAPI](https://fastapi.tiangolo.com/tutorial/testing/)
<https://fastapi.tiangolo.com/tutorial/testing/>